ENGINEERING FEASIBILITY STUDY / 20 OCTOBER 2026

Avatar kinematics and paired-view robot learning

Decision: proceed with the capture pipeline and a small policy experiment. Treat general video-to-robot transfer and learned-world-model RL as later research gates.

Prepared for Winston Smith. Reviewed PraxisOG/avatar-world-model at commit 3582489, including all five files. The current v2 thesis makes the dataset and capture pipeline the deliverable. The repository contains no implementation, tests, trained models, or collected episodes. [R]

Deliverable	Feasibility on your setup	Main condition
Synchronized sim views + state/action export	High; ordinary engineering	Explicit timing, calibration and reproducible replay
Useful small public dataset	Moderate to high	Diversity, clean rights and a demonstrated learning benefit
One SO-101 tabletop policy	Moderate	Real calibration, real demos and a narrow task
Exo video to useful ego representation	Research risk	Beat a direct exo-to-action baseline
Arbitrary cooking video to robot skill	Low confidence in current scope	Missing ground truth, human/robot mismatch and hidden contacts
RL inside a learned visual world model	Defer	Action-conditioned dynamics and validated counterfactual rollouts

What makes this worth doing

Your manufacturing, printing and local-compute experience fits the capture and reproducibility work. A useful outcome is an inexpensive, documented dataset with exact simulated state/action labels and a small real-arm validation. That remains valuable if a generated-video model fails. However, useful data does not automatically become more valuable with age: its task coverage, licensing and measured downstream benefit determine its value.

Scope and confidence

This is a repository and primary-source feasibility review, not an execution benchmark. No access to Moe or a real arm was available, and no performance numbers below were measured on V620s. Cost and labor ranges are planning estimates. The scientific claims are separated from published author results and official platform documentation.

Recommended commitment: an initial 8-16 hours with existing hardware; approximately 60-120 active hours through a meaningful simulation baseline; a further 40-80 hours for a narrow real-arm pilot. Keep the first physical setup below $500.

1. Where the design needs correction

Exact labels do not remove the simulation-to-reality gap

The simulator can give exact commanded targets, actual joint positions, velocities and its own contact forces. Those are different quantities. A commanded target need not be reached, and simulated friction, compliance, backlash, latency and contact forces may differ from the physical arm. Replace the claim of no sim-to-real label gap with a precise claim: the simulated labels are known without visual estimation. Record command and response separately.

View translation, inverse dynamics and future prediction are separate tasks

A view translator estimates another camera image of the present scene. An inverse model estimates an action consistent with an observed transition. A world model must predict what happens under a chosen action, including actions absent from the demonstration. An exo-to-ego translator alone cannot provide the counterfactual environment needed for RL. Use the actual physics simulator for early perturbation and recovery experiments.

The hidden state is not uniquely recoverable

An occluded finger, wrist rotation, grip force or object surface can have several explanations consistent with the same pixels. More cameras and temporal context reduce ambiguity but do not guarantee uniqueness. Train with observation history, calibrated camera poses and a defined target embodiment. Preserve visibility masks and uncertainty. At deployment, use measured robot proprioception; do not ask generated pixels to reconstruct state you can read directly.

Replaying a hand skeleton does not reconstruct the demonstration

EgoDex supplies egocentric video and tracked human poses. Those do not by themselves supply a complete textured scene, moving object geometry, object trajectories, physical material parameters or the hidden contact state. Replaying the skeleton in a generic scene produces motion-derived synthetic data, not an exact missing external view of the original interaction. The proposed instant paired-data shortcut therefore contains a substantial reconstruction project. [9]

Human hands and a cheap gripper have different feasible actions

For the first experiment, keep the simulated and physical robot the same. A leader/follower pair controls one robot arm; it is not a bimanual robot. Later, human-to-arm transfer should target reachable end-effector motion, grasp events and object goals, with inverse-kinematics and collision checks. Arbitrary finger articulation and wrist trajectories cannot simply be mapped onto a small arm.

Prediction drift needs a baseline before an architecture choice

Start with short-horizon prediction of object pose and robot state, conditioned on actions; compare against persistence and the physics simulator. If a learned model is later used for planning, measure error at several horizons, action sensitivity, uncertainty and recovery outcomes under held-out disturbances. Photorealistic frames are not evidence of correct contact dynamics.

2. What the prior art actually establishes

Evidence	What is supported	What remains unproved here
Mirror Learning, 2026 [1]	Authors report perspective transfer and imitation learning; their lab shows CARLA-to-real robotaxi view synthesis.	Transfer to close-contact hand manipulation, cheap arms or your compute budget.
EgoExo-WM, 2026 [2]	Exo-derived poses and generated ego video improve an action-conditioned latent world model.	Its body-pose planning results do not validate dexterous robot control. Reported training uses 8 A40 GPUs.
EgoDemoGen, 2025/2026 [3]	Jointly transforms trajectories and images; evaluates downstream robot policies.	Approximately 600 real episodes were used with 4 H20 GPUs, not 4 V620s. Model: CogVideoX-5B-I2V.
AMD Real2Sim2Real, 2026 [4]	Author demonstration of Genesis, LeRobot, SmolVLA and a real Franka on ROCm.	The reported under-one-hour loop uses R9700/W7900 hardware. It is not an independent V620 reproduction.
robosuite [5]	Existing teleoperation plus stateful simulation and multiple robot/external cameras.	A paired-view simulator alone is not a new scientific capability.
Ego-Exo4D [6]	Real synchronized ego/exo recordings, cooking activities and body/hand annotations.	These human annotations are not robot motor commands or ground-truth contact forces.

The defensible contribution

A useful claim would be: a reproducible low-cost pipeline produces calibrated, synchronized observations, actions and privileged simulation labels; controlled experiments show which parts improve a policy on held-out views and on a cheap physical arm. The value is accessibility, data quality and measured transfer. Avoid the repository's broad claim that nobody has demonstrated a paired-view data factory: this review does not support priority or an exhaustive novelty claim.

Do not confuse validation of a direction with validation of this implementation

The cited research is real and supports trying the idea. It does not establish that a small synthetic dataset can recover manipulation-relevant hidden state from arbitrary human video. Published systems often rely on large pretrained models, specialized capture, substantial compute, filtering and carefully constrained tasks. Those dependencies belong in the feasibility calculation.

The cheapest falsifiable question

Does paired-view supervision improve a policy that has the same observations available at deployment? This can be answered with a small dataset and policy. It does not require pretraining a foundation video model, buying a humanoid or training RL inside an unvalidated neural simulator.

3. Software and compute on Moe

Stack	Assessment	Decision
Genesis core + rasterizer	Documented ROCm physics and OpenGL camera path. Headless rendering must be checked separately. [7]	Try one scene and two low-resolution cameras first.
Genesis Nyx / Madrona	Nyx repository requires NVIDIA CUDA 12.9+; Madrona camera backend is CUDA-only. Nyx install docs also disagree about packaging. [7,8]	Do not make either a V620 dependency.
MuJoCo / robosuite	CPU physics and offscreen rendering make a practical fallback; robosuite already has demonstration tools. [5,10]	Use if Genesis integration consumes the initial test budget.
Isaac Lab / Isaac Sim	Current Lab can use Newton without Isaac Sim, but documented GPU training is still CUDA; Isaac Sim rendering adds RTX requirements. [11]	The CPU-only idea does not solve this project's visual data workflow.
ManiSkill / SAPIEN	CPU simulation exists; AMD rendering support is platform-specific, while the main accelerated path targets NVIDIA. [12]	Possible later; no reason to port it for the first scene.
Gazebo	Potential route for a ROS-integrated robot; additional integration surface for this small experiment.	Not the first choice; no throughput assumption made.

Start with one GPU, then parallelize independent runs

Four approximately 32 GB GPUs are not one transparent 128 GB device. Data parallelism replicates weights; model sharding introduces communication and software work. ACT is a sensible first policy: LeRobot describes an approximately 80M-parameter model and supports AMD through a suitable PyTorch build. Its generic training guidance is not a V620 timing guarantee. [13,14]

Keep simulation/export and training in separate pinned environments. Check a forward/backward optimizer step, decoder behavior and numerical stability on one V620 before introducing fused kernels, compilation or multi-GPU training. Use separate GPUs for seeds or data jobs first. Your existing vLLM workload will also compete for memory and time.

Measurements that resolve the actual risk

Record simulation steps/sec, two-camera frames/sec, encoding throughput, dropped frames, sample-load time, training steps/sec and peak VRAM. Measure a warm steady-state 500-1,000-step training run and extrapolate from that. Begin with native ROCm rather than adding a CUDA translation dependency. AMD lists V620/gfx1030 support, but that does not certify every Python extension or renderer. [15]

4. Data contract and release requirements

LeRobot is a good consumer-facing format. Its v3 layout combines Parquet signals, MP4 camera streams and episode metadata. Pin the exporter and reader versions, and prove a load/train round trip. Keep raw timing and calibration in a companion manifest rather than silently assuming the standard training schema captures every acquisition detail. [16]

Record	Required meaning
Time and transition	Episode ID, step index, simulation time, raw device timestamps and intended control interval. Observation at t; action applied over t to t+1; resulting state at t+1.
Action	Desired joint targets or deltas with units, ordering, reference frame, control mode, gripper semantics and duration. Preserve operator input separately if it differs.
Measured state	Actual q, velocity, gripper state, end-effector pose and measured telemetry. Commanded position must not be relabeled as measured position.
Both views	Frames from the same state, camera intrinsics/distortion, image dimensions and world/link extrinsics. Identify fixed, wrist or head camera explicitly.
Privileged labels	Object poses, segmentation, depth, contact events and simulated forces. Mark as simulation-only; exclude from ordinary deployment policy inputs.
Provenance and outcome	Asset versions and rights, seeds, simulator/driver/exporter versions, task, success/failure/intervention labels and checksums.

Synchronization must be designed, not inferred from matching frame counts

In simulation, freeze one state while rendering both cameras; never advance physics between views. Define whether images are captured before or after applying each command. In reality, two consumer USB cameras are not hardware-synchronized. Preserve acquisition timestamps and estimate lag; quantify residual skew instead of claiming exact alignment. At 30 Hz, one frame is 33.3 ms; at a modest 0.3 m/s gripper speed that is about 10 mm of motion.

Mixed frame rates and camera intrinsics

Keep original streams and export a declared common control grid, initially 10 or 15 Hz if real cameras are the bottleneck. Explicitly integrate, sample or hold actions according to their meaning. Upsampling 10 Hz data to 30 Hz creates no new evidence. Intrinsics, crop/resize transforms and camera placement are model inputs or metadata, not interchangeable after concatenation.

A public dataset people can trust

Publish task definitions, a data card, replay/export scripts, frozen splits, asset attribution, known failure modes and baseline configs. Split by episode and underlying trajectory before re-rendering; all views and augmentations of one trajectory stay together. Do not train on a third-party test split. Keep restricted data separate. EgoDex uses NC/ND terms; do not assume transformed examples may be redistributed as an openly licensed dataset. A format conversion alone is different from generating derivatives. [9,17]

5. Experiments that can justify continuing

Minimum useful environment

One SO-101-equivalent arm, one parallel gripper, a table, a few rigid objects and two calibrated camera mounts. Start with reaching/pushing, then pick-and-place into a large bin. Use keyboard/gamepad teleoperation or a scripted controller before buying a leader arm. A wrist camera is useful for robot control but does not, by itself, validate a human head-camera transformation.

Controlled policy comparison

Experiment	What it tests
A: exo + proprioception to actions	Direct baseline. Establish success without a generated ego image.
B: A plus paired-view latent alignment	Whether a training-only second view improves the shared representation.
C: A plus ego image prediction loss	Whether dense pixel supervision adds useful information beyond B.
D: exo to generated ego to actions	Whether the full proposed translation intermediate helps or compounds error.
Oracle: real/rendered ego + proprioception	An upper reference using the actual second view; keep separate from fair same-input comparisons.

Use identical underlying trajectories and held-out splits, comparable policy capacity and explicit training-time/compute budgets. Count auxiliary-head and generator costs. B and C must not receive the true second view during deployment if A and D cannot. Report three training seeds and task success, intervention rate, contact failures and action latency; reconstruction metrics are secondary.

Data volume and acceptance gates

For a pilot, target 100-300 distinct 20-40 second episodes across three tasks, including labeled failures, occlusions and recoveries. This is approximately 0.6-3.3 hours of unique motion, not multiplied by camera count. First overfit a few episodes to test the implementation; then evaluate on whole held-out layouts, objects and camera poses.

Suggested engineering gates, not published standards: replay/export passes every alignment check; the direct policy reaches at least 80% success on 50 held-out simulation trials per seed; paired supervision improves a preselected held-out condition by at least 10 percentage points across seeds without harming the basic task. If differences are noisy, expand evaluation and report confidence intervals rather than declaring a win.

Replace the unscorable cooking acceptance test

Use a synchronized Ego-Exo4D cooking subset for view/pose evaluation, respecting its access terms and annotation coverage. Use new paired robot captures to score real robot actions.
EPIC-KITCHENS-100 is egocentric, so it cannot alone supply the external-to-internal paired ground truth proposed in the repo. Ordinary internet cooking clips can remain qualitative demonstrations, not a final quantitative pass/fail test. [6,18]

Separate the human benchmark from the robot benchmark. Passing arm pick-and-place says little about human dexterity; producing convincing human-view video says little about controlling a real gripper.

6. Cost, workload and a realistic first release

Hardware spending after the simulation gate

The official SO-101 source BOM lists about $121.94 for one follower and $229.88 for a leader/follower pair, before printing and location-dependent costs. These are maintainer BOM estimates, not a delivered checkout quote. A pair provides one teleoperated manipulator. [19]

Item	Planning allowance	Basis
Follower + existing input device	$180-300 all-in target	Cheapest route; printed parts, simple RGB camera(s), mounting and freight vary.
Leader/follower workbench	$350-500 total target	~$230 BOM + $20-40 print materials + $30-70 RGB cameras + $20-40 extras + $50-120 tax/freight/contingency.
Simulation-only pilot	$0 hardware	Use Moe, existing storage and simple assets.
Depth, tactile, dexterous hands	$0 committed initially	None is necessary to test the core paired-view hypothesis.

Treat $500 as a cap, not a shopping target: if the delivered pair exceeds it, start with a follower and existing input device. Your printing capability is useful here. Do not budget a force-controlled dexterous rig from the repo's unverified hand-price claims; that is outside this feasibility gate.

Labor estimates: active engineering time, excluding unattended training

Gate	Hours	Exit criterion
0. Platform check	8-16	Two synchronized views, recorded state/action, one training optimizer step; fallback if blocked.
1. Capture and export	20-40	Reproducible scene, teleop/scripted demos, timestamps, calibration and LeRobot load round trip.
2. Dataset and baselines	32-64	100-300 episodes; held-out direct baseline; initial paired-view ablation.
3. Real-arm pilot	40-80 additional	Calibrated arm, real-only baseline and mixed-data comparison; target at least 20/30 successful narrow-task trials.
4. Shareable release	8-16 additional	Pinned environment, dataset card, provenance, split manifests and reproducible result.

Total: 60-120 hours for a useful simulation result; 108-216 hours through the real pilot and release. At an assumed 8 hours/week, that is roughly 2-4 months for simulation and 3-6 months for the complete narrow release. These are estimates for integrating existing components; broad cooking transfer has no credible completion date yet.

Storage and electricity are manageable if you stream data

Two uncompressed 640x480 RGB streams at 30 Hz use about 199 GB/hour; at an assumed 2-6 Mb/s each, compressed video uses 1.8-5.4 GB/hour before caches and backups. Four 150 W GPUs use 14.4 kWh per fully loaded day: $2.16-4.32 at an illustrative $0.15-0.30/kWh, plus the rest of the server. These are arithmetic estimates.

7. Decisions for the repository's open questions

README question	Recommended answer
1. Simulation stack	Genesis core/rasterizer on a strict initial time budget; MuJoCo fallback. Verify rendering independently of ROCm physics. Do not depend on Nyx or Madrona on V620.
2. Pixel ambiguity	Yes, proprioception must be first-class. Add camera geometry, temporal context, visibility/uncertainty and privileged simulation labels. A second camera does not identify hidden force.
3. Prediction drift	Use real simulation for recovery data now. Validate a small action-conditioned state/latent predictor before video rollouts or learned-model RL.
4. Translation intermediate	An experiment, not an assumption. Compare direct action prediction, latent alignment, pixel auxiliary loss and the full generated-ego path.
5. Episode format	Use a pinned LeRobot exporter plus raw timestamps/calibration manifest. Normalize rates and action semantics explicitly.
6. Cheap-arm transfer	One follower first if needed; position-controlled rigid-object tasks. Establish real-only success, then test whether synthetic data helps.
7. Dataset value	Evidence of learning benefit, diverse independent trajectories, rights to assets/data, frozen splits and easy reproduction matter more than raw frame count.

The highest-risk assumptions and their stop conditions

Platform risk: if two-camera capture or one-GPU training cannot be made stable within 8-16 focused hours, reduce dependencies or use MuJoCo; do not buy GPUs to rescue an untested hypothesis.

Scientific risk: if paired-view objectives do not beat the direct baseline under controlled evaluation, publish the negative result and useful capture tooling. Stop expanding the translation model until there is evidence that it helps.

Transfer risk: if a real-only policy cannot perform the simple task, fix calibration, timing, mechanics or demonstrations before blaming synthetic data. Compare real-only with real-plus-sim at the same real-data budget.

Coverage risk: if performance collapses under a changed camera or object placement, add those variations and investigate shortcuts before adding thousands of near-duplicate frames.

Recommended next action

Create one reproducible benchmark scene and a capture contract. Record a short two-camera episode with distinct observation/state/action fields, reload it through LeRobot, and train a tiny baseline on one V620. This is enough to test the tooling and reveal the next bottleneck. Keep human-avatar reconstruction, passive-video pretraining and learned-world-model RL behind separate evidence gates.

No repository files were edited and no issues or PRs were posted as part of this study. The design remains at the reviewed commit. All proposed acceptance thresholds and cost/labor estimates are reviewer judgments, not guarantees.

Sources and verification trail

Primary sources reviewed on 20 October 2026. Numbered references identify the claims above. Author papers and vendor/maintainer documentation are evidence of reported behavior; this study did not independently reproduce their experiments. Links are clickable.

[R] PraxisOG, avatar-world-model. All five files at 35824898a9220405ffa6362ec4bff810c02969f3.

[1] Liu et al., Mirror Learning (2026); author laboratory explanation and sim-to-real examples.

[2] Tran et al., EgoExo-WM: Unlocking Exo Video for Ego World Models (2026), appendix A.1.

[3] Xu et al., EgoDemoGen (2025; revised 2026), appendices B/C and generation efficiency.

[4] Yang et al., Real2Sim2Real for Vision-Language-Action Manipulation: An AMD ROCm-Based Pipeline (2026).

[5] robosuite: human demonstration collection and multiple-camera environment configuration.

[5a] robosuite RobotEnv camera_names, eye_in_hand and offscreen rendering options.

[6] Ego-Exo4D overview: synchronized views, activities, body and hand annotations.

[7] Genesis installation and camera sensors: ROCm backend, rasterizer and CUDA-only batch cameras.

[7a] Genesis installation: platform support and headless rendering.

[8] Genesis Nyx repository: public wheel requirements, NVIDIA GPU and CUDA 12.9+.

[8a] Genesis Nyx guide: conflicting internal-index installation note.

[9] Apple EgoDex repository: data contents, train/test split, downloads and dataset license.

[10] MuJoCo rendering documentation: headless graphics and renderer separation.

[11] Isaac Lab 3.0 installation: Newton, CUDA training and optional Isaac Sim/RTX requirements.

[12] ManiSkill installation: CPU/GPU simulation and platform-dependent rendering support.

[13] Hugging Face LeRobot ACT documentation: architecture and training guidance.

[14] LeRobot PyTorch accelerators: AMD via the PyTorch CUDA-facing device API.

[15] AMD ROCm system requirements; V620/gfx1030 listed. Also consult the current compatibility matrix.

[15a] AMD current ROCm compatibility matrix; align the actual installed stack rather than relying on a generic support label.

[16] LeRobotDataset v3.0: Parquet, MP4, metadata and temporal sampling.

[17] Creative Commons BY-NC-ND 4.0 deed: redistribution restrictions and format-conversion distinction.

[18] EPIC-KITCHENS-100 official dataset description: head-mounted egocentric kitchen recordings.

[19] TheRobotStudio SO-ARM100/SO-101 official BOM: one follower and leader/follower costs.